EXPERIMENT 7 - MODEL PLANNING AND BUILDING - CLUSTERING

Learning Objective

The objective of this experiment is to understand and implement the **K-Means clustering algorithm** for customer segmentation. The experiment involves analysing customer Annual Income and Spending Score, standardizing the selected features, determining the optimal number of clusters using the Elbow Method, building the K-Means model, visualizing customer segments, and analysing the characteristics of each cluster.

Libraries Used / Purpose

- **Pandas** – Used to load, explore, clean, and manipulate the customer dataset.
- **NumPy** – Used for numerical operations.
- **Matplotlib** – Used to create scatter plots and the Elbow Curve.
- **Seaborn** – Used for statistical visualization and pair plots.
- **Scikit-learn StandardScaler** – Used to standardize Annual Income and Spending Score.
- **Scikit-learn KMeans** – Used to perform K-Means clustering

Program Code

```
!pip install -q kagglehub

import kagglehub import
pandas as pd import numpy
as np import
matplotlib.pyplot as plt import
seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
sns.set_theme(style="whitegrid")

path = kagglehub.dataset_download(
    "vjchoudhary7/customer-segmentation-tutorial-in-python"
)

print("Dataset path:", path)
```

```
import os

files = os.listdir(path) print("Files:",
files)

csv_file = [
os.path.join(path, file)
for file in files     if
file.endswith(".csv")
][0]

df = pd.read_csv(csv_file)

print("\n--- First 5 Records ---") display(df.head())

print("\n--- Last 5 Records ---") display(df.tail())

print("\n--- Dataset Shape ---") print(df.shape)

print("\n--- Data Types ---") print(df.dtypes)

print("\n--- Missing Values ---")
display(df.isnull().sum())

print("\n--- Duplicate Records ---")
print(df.duplicated().sum())

print("\n--- Summary Statistics ---")
display(df.describe())

features = [     'Annual
Income (k$)',
    'Spending Score (1-100)'
]

X = df[features].copy()
                Selected Features ---")
display(X.head())

plt.figure(figsize=(8, 6))

sns.scatterplot(     data=df,
x='Annual Income (k$)',
y='Spending Score (1-100)'
)
```

```
print("\n---


plt.title('Annual Income vs Spending Score')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)') plt.show()

sns.pairplot(
df[features]
)

plt.show()

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(
X_scaled,     columns=features
)

print("\n--- Standardized Features ---")
display(X_scaled_df.head())

inertia = []

K_range = range(1, 11)

for k in K_range:

   kmeans = KMeans(
n_clusters=k,
random_state=42,         n_init=10
   )

   kmeans.fit(X_scaled)

   inertia.append(kmeans.inertia_)
plt.figure(figsize=(8, 6))

plt.plot(
K_range,
inertia,
marker='o'
)

plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia') plt.xticks(K_range)
plt.show()

optimal_k = 5
```

```
kmeans = KMeans(
    n_clusters=optimal_k,
random_state=42,      n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

centroids_scaled = kmeans.cluster_centers_

centroids_original = scaler.inverse_transform(
centroids_scaled
)

centroid_df = pd.DataFrame(
centroids_original,      columns=features
)

centroid_df['Cluster'] = range(optimal_k)

centroid_df = centroid_df[
    [
        'Cluster',
        'Annual Income (k$)',
        'Spending Score (1-100)'
    ]
]

print("\n--- Cluster Centroids ---")
display(centroid_df)
                Dataset with Cluster Labels ---")
display(df.head(10))

output_file = "mall_customers_clustered.csv"

df.to_csv(
output_file,
index=False
)

print("\nClustered dataset saved as:", output_file)

plt.figure(figsize=(10, 7))

sns.scatterplot(      data=df,
x='Annual Income (k$)',
y='Spending Score (1-100)',
hue='Cluster',
palette='deep',      s=80
```

```
print("\n---


)

plt.scatter(     centroid_df['Annual
Income (k$)'],     centroid_df['Spending
Score (1-100)'],     marker='X',     s=250,
color='black',     label='Centroids'
)

for _, row in centroid_df.iterrows():

    plt.annotate(
f"C{int(row['Cluster'])}\n"
f"({row['Annual Income (k$)']:.1f}, "
f"{row['Spending Score (1-100)']:.1f})",
        (
            row['Annual Income (k$)'],
row['Spending Score (1-100)']
        ),
        xytext=(8, 8),
textcoords='offset points'
    )

plt.title('Customer Segmentation using K-Means')
plt.xlabel('Annual Income (k$)') plt.ylabel('Spending
Score (1-100)')
plt.legend() plt.show()

cluster_counts = (
df['Cluster']
.value_counts()
    .sort_index()
    .reset_index()
)

cluster_counts.columns = [
    'Cluster',
    'Number of Customers'
]

print("\n--- Number of Customers in Each Cluster ---") display(cluster_counts)

cluster_analysis = (
df.groupby('Cluster')[
        [
            'Annual Income (k$)',
            'Spending Score (1-100)'
        ]
    ]
    .mean()
```

```
    .reset_index()
)

print("\n--- Average Income and Spending Score by Cluster ---") display(cluster_analysis)

cluster_analysis = cluster_analysis.merge(
cluster_counts,     on='Cluster'
)

print("\n--- Complete Cluster Analysis ---")
display(     cluster_analysis[
        [
            'Cluster',
            'Number of Customers',
            'Annual Income (k$)',
            'Spending Score (1-100)'
        ]
    ]
)
                Customer Segment Interpretation ---")

for _, row in cluster_analysis.iterrows():

    income = row['Annual Income (k$)']
spending = row['Spending Score (1-100)']
cluster = int(row['Cluster'])

    if income >= 70 and spending >= 60:
        segment = "High Income - High Spending"
        strategy = "Premium products, loyalty rewards and personalized offers."

    elif income >= 70 and spending < 40:
        segment = "High Income - Low Spending"          strategy = "Discount campaigns,
product recommendations and targeted promotions."

    elif income < 40 and spending >= 60:
        segment = "Low Income - High Spending"          strategy = "Affordable
products, frequent promotions and loyalty incentives."

    elif income < 40 and spending < 40:
        segment = "Low Income - Low Spending"         strategy = "Budget
products and low-cost promotional campaigns."

else:
        segment = "Average Income - Average Spending"         strategy = "General
promotions, cross-selling and personalized recommendations."
     print(         f"Cluster {cluster}:
{segment} | "         f"Marketing
Strategy: {strategy}"
```

```
print("\n---


)
```

Output

```
--- First 5 Records ---

   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40


       CustomerID         Age  Annual Income (k$)  Spending Score (1-100) count
200.000000  200.000000          200.000000             200.000000 mean
100.500000   38.850000          60.560000              50.200000 std
57.879185   13.969007           26.264721              25.823522 min
1.000000   18.000000           15.000000              1.000000 25%     50.750000
28.750000          41.500000              34.750000
50%    100.500000   36.000000           61.500000              50.000000
75%    150.250000   49.000000           78.000000              73.000000
max    200.000000   70.000000          137.000000              99.000000
```